
MAFS5440 Final Project: M5 Forecasting

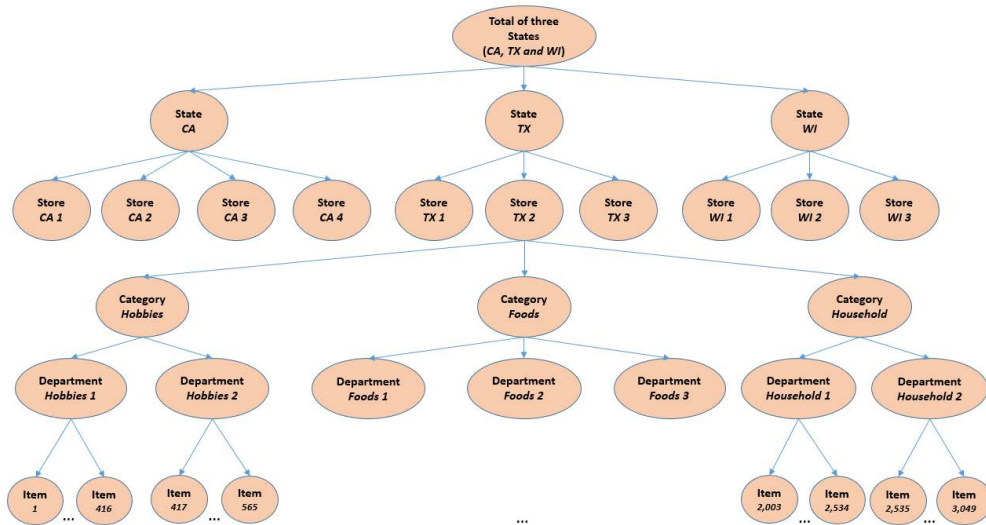
Bao Shijie (21002219)
Chen Jianfei (21074951)
Yu Xiaoping (21023419)
Gu Jingjia (21011777)
Department of Mathematics (MAFM)
Hong Kong University of Science and Technology

1 Introduction

The M5 competition aims to forecast daily sales for the next 28 days(till 22nd May 2016), and to make uncertainty estimates for these forecasts, utilizing historical daily sales data from Walmart stores. In this final project, we join both accuracy and uncertainty competition. We perform feature engineering, recursive and non-recursive LightGBM models for accuracy prediction, generating uncertainty prediction from accuracy results. We find that simple but practical model can produce very good results.

2 Data

The M5 dataset, involves the unit sales of various products sold in the USA, organized in the form of grouped time series. More specifically, the dataset involves the unit sales of 3,049 products, classified in 3 product categories (Hobbies, Foods, and Household) and 7 product departments, in which the above-mentioned categories are disaggregated. The products are sold across ten stores, located in three States (CA, TX, and WI).



The historical data range from 2011-01-29 to 2016-06-19, which means the products have a selling history of 1,941 days(test data of h=28 days not included). Our raw datafiles are as follows:

- sales_train_evaluation.csv
- sales_train_validation.csv
- sell_prices.csv
- calendar.csv
- sample_submission.csv

As for the forecasting horizon, the number of forecasts required, both for point and probabilistic forecasts, is $h=28$ days (4 weeks ahead). For better explanation, the train-val-test split is shown as follows:

- Training set: 2011-01-29 to 2016-04-24 (d_1 to d_1913)
- Validation set: 2011-04-25 to 2016-05-22 (d_1913 to d_1941)
- Testing set: 2011-05-22 to 2016-06-19 (d_1941 to d_1969)

where 'd_i' follows the form of column 'd' in dataset 'calendar.csv'

3 Feature Engineering

In our project, we dedicated a significant amount of time and energy to feature engineering, recognizing its critical role in enhancing model performance. We meticulously analyzed the data, exploring various transformations and interactions to create meaningful features that could capture underlying patterns.

In conclusion, there are 5 categories of features applied in our project: sales features, price features, calendar features, lag features, mean encoding features.

3.1 Sales Features

Actually there are not many new features in this part, but this part builds the base for our feature engineering. We use the dataset 'sales_train_validation.csv'. After we read the csv file, we find that there are only 30490 rows for training, while we could have more information for each day. Therefore, we 'melt' the original dataframe, transforming from the horizontal view to vertical grid. To be more specific, Indexes are 'id', 'item_id', 'dept_id', 'cat_id', 'store_id', 'state_id', with 'd_' columns as labels. We successfully enlarge the rows from 30490 to 58327370.

After transformation, we find too many 0 in the column 'sales' of each row. While there are two reasons can lead to 0 in our dataset: zero sales and absence of item in the store, and we only need the former one. So we calculate the release date of each item and many redundant rows are filtered. Finally our train dataset has 46881677 rows. After building the basis dataframe, all features generated later are merged to this dataframe.

3.2 Price Features

We use the dataset 'sell_prices.csv' to generate features regarding to price. We first apply some basic aggregations(max,min,mean,std). We also normalize the sell price by maximal sell price to generate new feature. Moreover, we believe that momentum effect is hidden in price data. We generate momentum features using shifting by week, month mean and year mean.

3.3 Calendar Features

We use the dataset 'calendar.csv'. As for the calendar features, in addition to original columns, we believe that data can hold much information, so we make some features from date such as day of the week, week of the month and whether weekend.

3.4 Lag Features

In this part, we use the processed dataframe after generating sales features. We create lag features to reflect past sales and rolling mean and std features to reflect historical trend. Lag days are chosen as 1-14 and Rolling windows are chosen as 7,14,30,60,180.

3.5 Mean Encoding Features

In the last part, we also use the processed dataframe after generating sales features. We believe that id columns should have much information, so we encode id columns to generate our features.

3.6 Features

Below is the overview of features we applied in our project:

Type	Column name
sales features	sales
price features	sell price price aggregations(max,min,mean,std) price_norm price nunique price momentums(week,month,year)
calendar features	event related features day,week,month,month,year day of week, week of month whether weekend
lag features	sales lag features(1-14) rolling mean, std features(7,14,30,60,180)
mean encoding features	enc_store_id_dept_id_mean enc_store_id_dept_id_std enc_item_id_state_id_mean enc_item_id_state_id_std

4 Accuracy

4.1 Modeling Strategy

As for our approach to forecast sales for two 28-day periods, our goal is building a simple solution that can be widely used in practice, since simple and practical features plus effective technique should lead to great results. The technique we use is LightGBM model only.

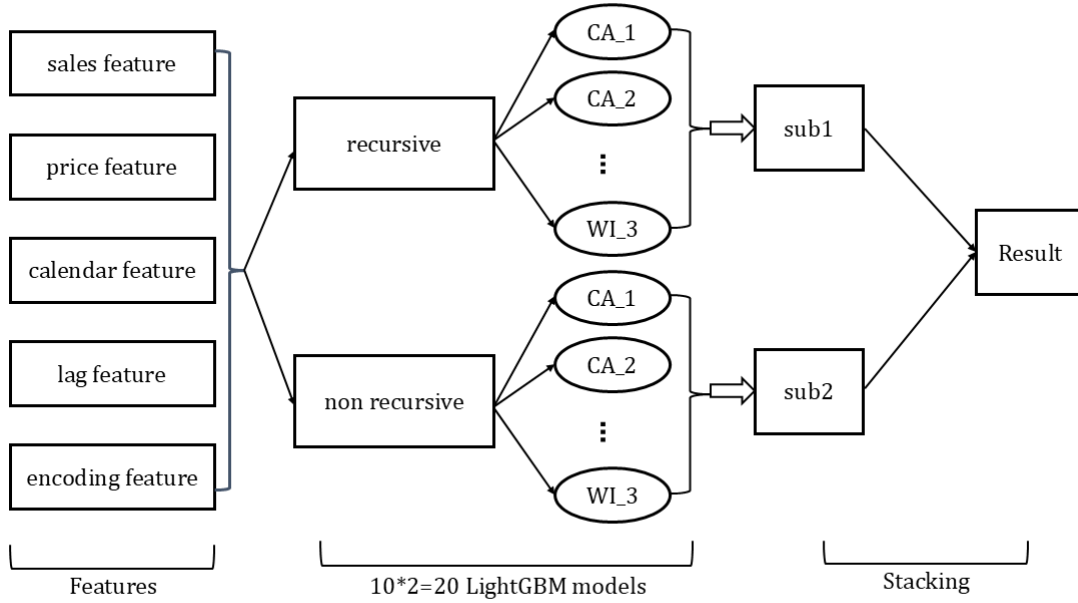
To introduce our modeling strategy in detail, we apply recursive and non-recursive strategy separately, and we ensemble the two part as our final model strategy. For each strategy, we divide the training data into groups by store_id, and apply LightGBM for each group to train and predict. As there are 10 stores in dataset, we have 10 models for each strategy. Therefore, we have 10*2=20 LightGBM models for our forecasting approach. We divide training data into groups by store_id for memory efficiency.

The flow chart explaining our approach is as follows:

4.2 LightGBM

LightGBM, proposed by Microsoft in 2017, is another evolutionary version of the GBDT model. LightGBM is a lightweight gradient lifting machine (GBM). Therefore, in the face of large-scale data sets, LightGBM is still very calm and lighter. LightGBM has made many optimizations on the basis of XGBoost, it continues the ensemble learning method of XGBoost, but it pays more attention to the training speed of the model. Considering the advantages, we choose LightGBM as our only model.

It's worth mentioning that our LightGBM model uses Tweedie loss objective function, we decide to apply Tweedie objective function after learning from Kaggle community. The parameters we use in our LightGBM model is shown as follows:



```

lgb_params = {
    'boosting_type': 'gbdt',
    'objective': 'tweedie',
    'tweedie_variance_power': 1.1,
    'metric': 'rmse',
    'subsample': 0.5,
    'subsample_freq': 1,
    'learning_rate': 0.015,
    'num_leaves': 2**8-1,
    'min_data_in_leaf': 2**8-1,
    'feature_fraction': 0.5,
    'max_bin': 100,
    'n_estimators': 3000,
    'boost_from_average': False,
    'verbose': -1,
    'seed': 1995
}

```

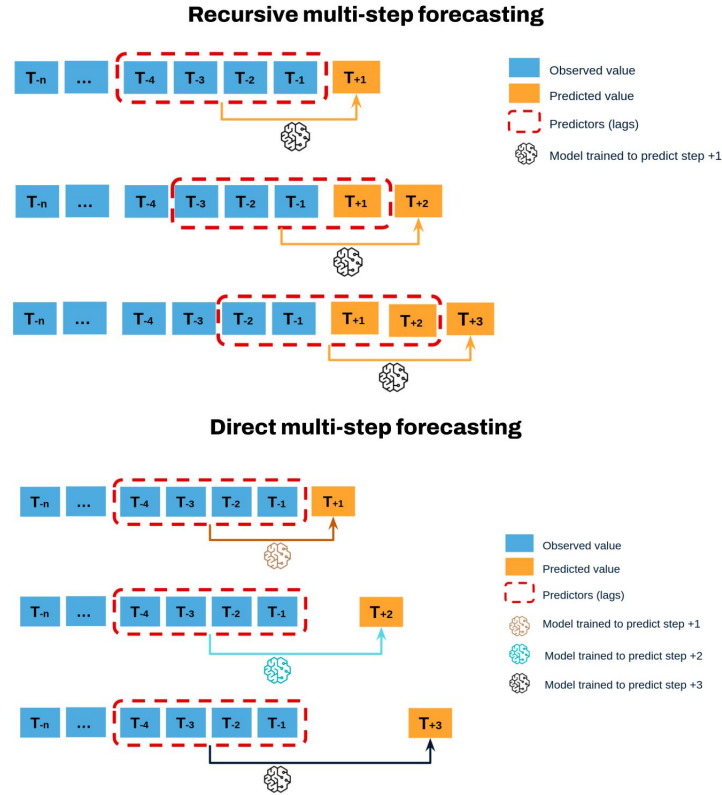
4.3 Recursive and Non-recursive Strategy

In our approach, we apply both recursive and non-recursive strategy to our forecasting task.

For recursive model, The model is trained in a loop for each store, and predictions are made recursively for each day (i.e., the model uses the predictions from previous days to inform future predictions).

For non-recursive model, The model is trained on historical data for each store and validated without using previous predictions. The predictions are made for a fixed validation window without any feedback from earlier predictions.

The explanation of recursive and non-recursive model is as follows:



4.4 Ensembled Result

For recursive model, the error is accumulated and it leads to large variation. For non-recursive model, it's based solely on historical data without the influence of prior predictions, resulting in a more straightforward but potentially less responsive model. Therefore, we decide to ensemble the result of recursive model and non-recursive model. Finally we got the result shown as follows:

Submission and Description	Private Score 📄	Public Score 📄	Selected
 submission_final.csv Complete (after deadline) · 1d ago · ensembled model	0.55095	0.38660	☐

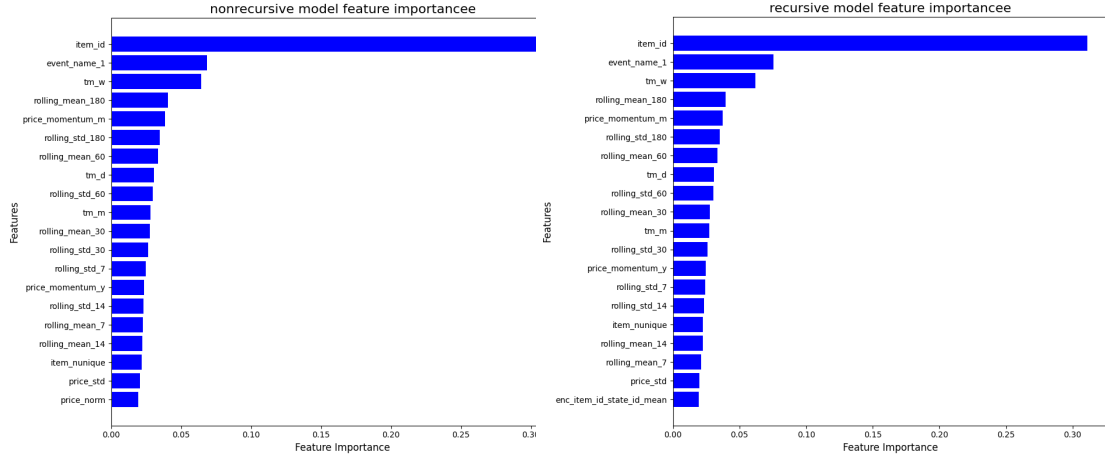
4.5 Feature Importance

The feature importance of our recursive model and non-recursive model are as follows:

Analyzing the feature importance graphs, there are two categories of features that have high importance in our model:

- lags features: rolling mean, rolling std.
- calendar features: week, day, month, event.

Although price features and mean encoding features also contributes to the model, we can find that major features are mainly lags features and calendar features.



5 Uncertainty

5.1 Overview

In Uncertainty competition, we are required to predict 9 quantiles of item sales at stores in various locations for two 28-day time periods. To be more specific, we need to calculate the median and four prediction intervals. Raw data are the same as that provided in Accuracy competition.

After examining the sample submission data, we realized that we need to make quantile predictions for each level. The level 12 is the bottom-level, which is the only non-aggregated one. The level 1 is the top-level, which is the only all-aggregated one. The 12 levels are shown as follows:

Level id	Aggregation Level	Number of series
1	Unit sales of all products, aggregated for all stores/states	1
2	Unit sales of all products, aggregated for each State	3
3	Unit sales of all products, aggregated for each store	10
4	Unit sales of all products, aggregated for each category	3
5	Unit sales of all products, aggregated for each department	7
6	Unit sales of all products, aggregated for each State and category	9
7	Unit sales of all products, aggregated for each State and department	21
8	Unit sales of all products, aggregated for each store and category	30
9	Unit sales of all products, aggregated for each store and department	70
10	Unit sales of product x, aggregated for all stores/states	3,049
11	Unit sales of product x, aggregated for each State	9,147
12	Unit sales of product x, aggregated for each store	30,490
Total		42,840

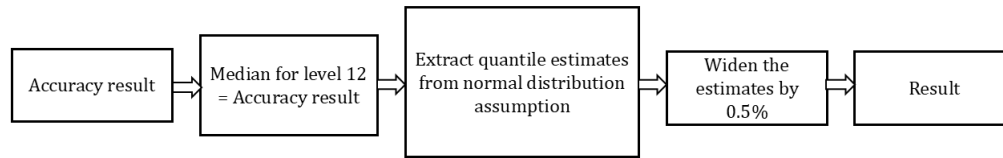
5.2 Workflow

To explain our workflow in detail, having the median from the Accuracy track as a starting point for level 12, with simple aggregations we obtain the median for the other 11 levels. Then we need to estimate the other 8 quantiles(4 prediction levels). We apply normal distribution in level 1-12, extracting quantile estimation from normal distribution.

We first assign a coefficient to different levels, then we map original quantiles to a set of new quantiles by the coefficient. We could calculate CDF of new quantiles in standard normal distribution. After normalizing we get the ratios for each level. The prediction interval is calculated by multiplying ratios and accuracy results.

The higher the aggregation level, the more confident we are in the point prediction. Therefore, we assign higher coefficient to bottom levels since we expect a wider prediction interval. Additionally, we know that normal distribution is a too strict assumption and we should give some space to our estimation. We finally widen our quantile estimates by 0.5%.

The flow chart below can explain out workflow in the uncertainty part:



5.3 Result

The kaggle result of our uncertainty estimation is as follows:

Submission and Description	Private Score ^①	Public Score ^①	Selected
 submission_uncertainty_final.csv Complete (after deadline) · 15s ago · Uncertainty final submission	0.22243	0.14297	☐

6 Further Improvements

There are several aspects we could explore if we have more time and computative power:

6.1 Accuracy

- Dividing training data in different ways: In our approach, we divide the training data by store_id. Similarly, we can divide the training data by cat_id and store_id, and even the combination of store_id and cat_id... There are many ways of combination, we can combine all the results to generate a more robust Accuracy prediction.
- Introduce weekly models: In our approach, every single LightGBM model is used to forecast the 28-day period. However, we can divide 28-day period into 4 weeks, and train a model for a single weekly period. In this way, we would have more LightGBM models and may have better results.
- Try other models: We have also seen many successful examples using other models such as LSTM and XGBoost. Maybe we can also apply these models to generate good result.

6.2 Uncertainty

- Customized distribution: In our approach, we use normal distribution assumption for all levels. However, it's natural some levels are suitable for other assumptions like skewed normal distribution. Our approach is still too simple.
- Tune the parameter: As for the coefficient assigned to each level, we tune the coefficients based on our experience and Kaggle community. If we could tune this parameter in a wiser way, we believe we would have better result.

7 Conclusion

7.1 Result Analysis

We have learned a lot during the process of completing the final project. Unlike what we thought before, that more complicated and more fancy model would definitely generate good results. The

technique we use in this project is quite simple, LightGBM only for Accuracy part and normal distribution for Uncertainty part, but it actually generates very good result. There are two key take-aways we learn from the final project:

- **Apply simple but practical model:** Although complicated would generate better results most of the time. We need to find the most simple but effective model for our problem instead of using fancy models without careful thoughts.
- **Learn from the Kaggle community:** During the process, we have gone through many discussions and experience from medal winners in Kaggle. Many of the ideas of feature engineering and model strategy are learned and combined from those excellent and kind Kaggle players. We want to express our sincere thanks to such a insightful community.

8 Contribution

Organization: Bao Shijie

Feature Engineering: Chen Jianfei, Bao Shijie

Accuracy: Yu Xiaoping, Bao Shijie

Uncertainty: Gu Jingjia, Bao Shijie

Report&Slides: Bao Shijie

References

Guolin Ke et al. Lightgbm: A highly efficient gradient boosting decision tree. In: Advances in neural information processing systems 30 (2017).

<https://www.kaggle.com/code/kyakovlev/m5-simple-fe>

<https://www.kaggle.com/code/kyakovlev/m5-lags-features>

<https://www.kaggle.com/code/kyakovlev/m5-custom-features>

<https://www.kaggle.com/code/kyakovlev/m5-three-shades-of-dark-darker-magic>